LLM Gateway — Architecture & Request Flow

Interview preparation · V1 · unified multi-provider gateway (FastAPI + PostgreSQL + React)

1. The Big Picture

What the project is and why it exists.

Q1. What is your LLM Gateway and what problem does it solve?

EASY

Answer: It is a single backend API that sits in front of multiple LLM providers (Gemini, Groq, and extensible to others). Clients call one endpoint, **POST /v1/chat/completions**, and the gateway handles authentication, validation, provider selection, calling the provider, normalizing the response, and logging usage.

Simple explanation: It is one front door to many LLMs, so the app never talks to Gemini or Groq directly.

Project example: The React console and any client send the same request shape with "model": "groq" or "model": "gemini"; the gateway does the rest.

Interview tip: Lead with the one-sentence value: 'one API, many providers, with central auth, logging and error handling.'

Q2. Why build a gateway instead of calling providers directly from the app?

EASY

Answer: Centralization. Without a gateway, every app would duplicate provider SDKs, store provider keys, re-implement error handling, and have no shared usage tracking. The gateway gives one place for auth, request validation, a common response shape, usage logging, and secret management.

Simple explanation: Do the cross-cutting work once in the middle instead of copy-pasting it into every client.

Project example: Provider API keys live only in the backend's .env; clients only hold a gateway key. Swapping a model is a config change in one place, not in every app.

Likely follow-ups:

- What do you lose by adding a gateway? (one extra network hop and a component to run)
- Is it a single point of failure? (yes; mitigated by it being stateless and horizontally scalable)

Interview tip: Name the concrete things centralized: keys, validation, logging, error shape.

---

2. Request Lifecycle

How one chat request travels through the system.

Q3. Walk me through the full flow of a chat completion request.

MEDIUM

Answer: The browser calls the FastAPI gateway. A request-ID middleware assigns one id. The route depends on authentication (gateway API key) and a DB session. Pydantic validates the body. The GatewayService resolves the public model id through the model registry, gets the right provider adapter from a factory, builds an internal request, and calls the adapter. The adapter calls the provider over httpx and converts the reply into our standard response. The route logs usage to PostgreSQL and returns the response with an X-Request-ID header.

Simple explanation: id -> auth -> validate -> pick model -> pick provider -> call it -> normalize -> log -> respond.

```
Browser
  -> FastAPI gateway (request-id, auth, validation)
  -> GatewayService
     -> model registry (alias -> provider + model)
     -> provider factory -> Gemini/Groq adapter -> provider API
  -> response normalization
  -> usage logging -> PostgreSQL
  -> response (+ X-Request-ID)
```

Interview tip: Memorize this chain; it is the backbone of almost every other answer.

Q4. Where does each cross-cutting concern live, and why there?

MEDIUM

Answer: Request IDs: middleware (runs for every request, before routes). Auth and DB session: FastAPI dependencies on the route. Validation: Pydantic schemas. Business orchestration: GatewayService. Provider specifics: the adapters. Error serialization: centralized exception handlers. Usage logging: a small usage service called from the route boundary.

Simple explanation: Each concern has exactly one home, so no logic is duplicated or tangled.

Project example: Logging lives at the route boundary (it has the identity, DB session, timing and final result) rather than inside GatewayService, which stays pure orchestration.

Likely follow-ups:

- Why not log inside GatewayService? (it would couple business logic to auth + DB)

3. Stack & Design Choices

The reasoning behind the main technology decisions.

Q5. Why FastAPI, and why async?

MEDIUM

Answer: FastAPI gives typed request/response models via Pydantic, automatic validation, built-in dependency injection, and auto-generated docs. It is async-native, which matters because the slow part of each request is a network call to an LLM. While one request waits on the provider, the worker can serve others.

Simple explanation: Typed, validated, async web framework that fits 'call a slow API and wait' perfectly.

Project example: Endpoints are async and providers use httpx.AsyncClient. The database uses synchronous SQLAlchemy, which FastAPI runs in a threadpool, so it does not block the loop.

Likely follow-ups:

- Why not Flask/Django? (no native async + less built-in validation/DI for this use case)
- Why sync SQLAlchemy in an async app? (tiny, infrequent queries; simpler Alembic; threadpool handles it)

Interview tip: Tie 'async' to the real reason: the provider call is the bottleneck.

Q6. How is the codebase organized to keep routes thin?

MEDIUM

Answer: Routes only validate, authenticate, trace, delegate, and log. The real work is pushed down: model resolution in the registry, orchestration in GatewayService, provider I/O in adapters, persistence in a repository layer, and error formatting in centralized handlers.

Simple explanation: The route is a thin coordinator; the heavy lifting lives in services and adapters.

Project example: The chat route is a few lines: get request id, time the call, delegate to gateway.complete(), log success/error, return. No provider logic at all.

Q7. What is intentionally OUT of scope for V1? EASY

Answer: No automatic retries, no provider fallback, no streaming, no rate limiting, no RBAC, no caching/Redis, no queues, no Kubernetes, and no intelligent model routing. The client explicitly picks the model.

Simple explanation: V1 is a solid, honest core; the 'smart' and 'scale' features are deliberately deferred.

Project example: The model registry maps aliases for OpenAI and Anthropic too, but only the Gemini and Groq adapters are implemented; requesting 'gpt' or 'claude' returns a clean 503.

Interview tip: Stating scope boundaries confidently signals engineering maturity, not gaps.

Q8. If you took this to higher scale (V2), what would you change and why? HARD

Answer: Add provider fallback and bounded retries with backoff for transient upstream errors; introduce caching for repeated prompts; add rate limiting per gateway key; possibly move to async DB drivers if DB load grows; add metrics/tracing; and consider streaming responses. Each is a deliberate addition on top of the clean V1 seams.

Simple explanation: V1's clean boundaries make these additions local, not rewrites.

Project example: Because GatewayService depends on the provider abstraction, retries/fallback can be added around the adapter call without touching routes or schemas.

Interview tip: Frame V2 as 'enabled by V1's design', not as 'fixing V1'.